Experiment No. 9

Object Detection and Recognition using Deep Learning Models on Standard Image Datasets

Aim

To implement a deep learning-based object detection and recognition system using a pre-trained model and evaluate its performance for identifying and localizing multiple objects.

Theory

Object detection is a computer vision task that combines object classification and object localization.

Unlike image classification, which only tells us what object is present, object detection:

Identifies the object Finds its location Draws a bounding box Assigns a class label Provides a confidence score

The experiment document specifically suggests pre-trained models such as YOLOv8, SSD, and Faster R-CNN, with datasets such as MS COCO, Pascal VOC, and Open Images.

```
# Install YOLOv8
!pip install ultralytics -q

# Import libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO

# ============================================================
# STEP 1: ENTER IMAGE PATH MANUALLY
# ============================================================

# If image is uploaded directly to Colab:
image_path = "/content/cars-city-traffic-daylight_23-2149092084.webp"

# Example:
# image_path = "/content/car.jpg"
# image_path = "/content/person.jpg"

print("Image path:", image_path)

# ============================================================
# STEP 2: LOAD PRE-TRAINED YOLO MODEL
# ============================================================

model = YOLO("yolov8n.pt")

print("YOLO model loaded successfully!")

# ============================================================
# STEP 3: PERFORM OBJECT DETECTION
# ============================================================

results = model(image_path)

# Get first result
result = results[0]

# ============================================================
# STEP 4: DISPLAY DETECTION RESULT
# ============================================================

result_image = result.plot()

plt.figure(figsize=(12, 8))
plt.imshow(cv2.cvtColor(result_image, cv2.COLOR_BGR2RGB))
```

```
plt.imshow(cv2.cvtColor(result_image, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("YOLO Object Detection")
plt.tight_layout()
plt.show()

# ============================================================
# STEP 5: DISPLAY DETECTED OBJECTS
# ============================================================

print("\n========== DETECTION RESULTS ==========")

if result.boxes is not None and len(result.boxes) > 0:

    class_ids = result.boxes.cls.cpu().numpy()
    confidences = result.boxes.conf.cpu().numpy()

    for i, (class_id, confidence) in enumerate(
        zip(class_ids, confidences), start=1
    ):
        class_name = model.names[int(class_id)]

        print(
            f"{i}. Object: {class_name} "
            f"| Confidence: {confidence:.2f}"
        )

else:
    print("No objects detected.")

# ============================================================
# STEP 6: COUNT OBJECTS
# ============================================================

print("\n========== OBJECT COUNT ==========")

if result.boxes is not None and len(result.boxes) > 0:

    detected_objects = {}

    for class_id in result.boxes.cls.cpu().numpy():
        class_name = model.names[int(class_id)]

        if class_name in detected_objects:
            detected_objects[class_name] += 1
        else:
            detected_objects[class_name] = 1

    for object_name, count in detected_objects.items():
        print(f"{object_name}: {count}")

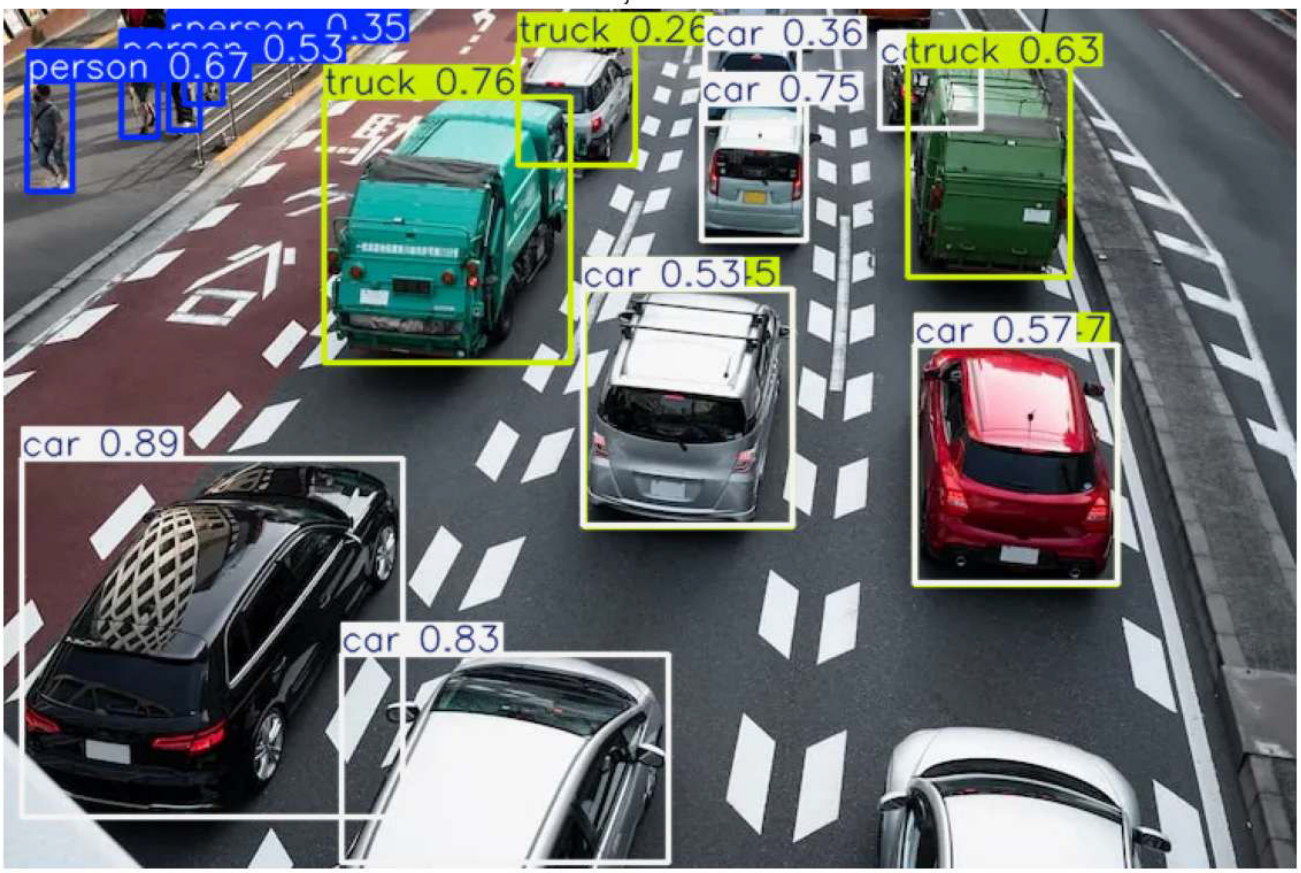
else:
    print("No objects detected.")

print("\nObject detection completed successfully!")
```

```
Image path: /content/cars-city-traffic-daylight_23-2149092084.webp
YOLO model loaded successfully!

image 1/1 /content/cars-city-traffic-daylight_23-2149092084.webp: 448x640 4 persons, 7 cars, 5 trucks, 151.2ms
Speed: 4.1ms preprocess, 151.2ms inference, 1.2ms postprocess per image at shape (1, 3, 448, 640)
```

YOLO Object Detection




```
========== DETECTION RESULTS ==========
1. Object: car | Confidence: 0.89
2. Object: car | Confidence: 0.83
3. Object: truck | Confidence: 0.76
4. Object: car | Confidence: 0.75
5. Object: person | Confidence: 0.67
6. Object: truck | Confidence: 0.63
7. Object: car | Confidence: 0.58
8. Object: car | Confidence: 0.57
9. Object: car | Confidence: 0.53
10. Object: person | Confidence: 0.53
11. Object: truck | Confidence: 0.47
12. Object: truck | Confidence: 0.45
13. Object: car | Confidence: 0.36
14. Object: person | Confidence: 0.35
15. Object: person | Confidence: 0.30
16. Object: truck | Confidence: 0.26

========== OBJECT COUNT ==========
car: 7
truck: 5
person: 4

Object detection completed successfully!
```

Questions and Answers

Q1. Differentiate between image classification, object detection, and image segmentation.

Answer:

Technique	Description
Image Classification	Identifies the main class/category of an image
Object Detection	Identifies objects and their locations using bounding boxes
Image Segmentation	Assigns a class to individual pixels/regions

Example: For an image containing a dog and a car:

- **Classification** → "Dog and car"
- **Detection** → Dog and car with bounding boxes
- **Segmentation** → Exact pixels belonging to dog and car

Q2. Explain the working principle of YOLO.

Answer:

YOLO stands for **You Only Look Once**.

YOLO processes the entire image through a neural network in a single forward pass. It predicts:

- Bounding boxes
- Class labels
- Confidence scores

The predictions are then filtered to remove low-confidence detections and overlapping boxes.

The main advantage is that YOLO performs detection very quickly, making it suitable for real-time applications.

Q3. Compare YOLO, SSD, and Faster R-CNN.

Answer:

Feature	YOLO	SSD	Faster R-CNN
Speed	Very high	High	Lower
Accuracy	High	High	Very high
Real-time use	Excellent	Good	Limited
Architecture	Single-stage	Single-stage	Two-stage
Application	Real-time detection	Real-time detection	High-accuracy detection

The experiment specifically asks comparison in terms of speed, accuracy and practical applications.

Q4. What is a bounding box? Why is it important?

Answer:

A bounding box is a rectangular box drawn around a detected object.

It is generally represented using coordinates:

$$(x1, y1, x2, y2)$$

It is important because it tells us where the object is located in the image.

Q5. Explain confidence score and IoU.

Answer:

Confidence Score

The confidence score represents how confident the model is that a detected object belongs to a particular class.

For example:

Person → 0.92

means the model has a confidence of 92%.

IoU

IoU stands for Intersection over Union.

It measures the overlap between the predicted bounding box and the ground-truth bounding box.

$$IoU = \frac{Area\ of\ Intersection}{Area\ of\ Union}$$

A higher IoU means better overlap.

The experiment specifically requires analysis of both confidence score and IoU.

Q6. What are the advantages of pre-trained deep learning models?

Answer:

Advantages include:

- Less training time
- Less computational requirement
- Good performance
- Can detect multiple objects
- Useful features are already learned
- Easy to fine-tune for new datasets
- Better than many traditional hand-crafted image-processing methods

Q7. Why are MS COCO and Pascal VOC widely used?

Answer:

MS COCO and Pascal VOC are widely used because they provide:

- Large collections of labeled images
- Multiple object categories
- Bounding-box annotations
- Standard evaluation methods
- Common benchmarks for comparing object-detection models

These datasets are specifically mentioned in the experiment sheet.

Q8. Mention five real-world applications.

Answer:

Five applications are:

- Autonomous vehicles
- Security and surveillance
- Healthcare
- Retail analytics
- Industrial inspection

The experiment sheet also identifies autonomous driving, surveillance, retail analytics and industrial inspection as example applications.

Q9. What challenges occur in complex real-world environments?

Answer:

Common challenges include:

- Poor lighting
- Object occlusion
- Small objects
- Similar-looking objects
- Complex backgrounds
- Different object sizes
- Fast-moving objects
- Blurred images
- Crowded scenes

- Low-quality images

Q10. How can object detection be improved?

Answer:

Object detection can be improved using:

- Larger and better-quality datasets
- Data augmentation
- Improved neural-network architectures
- Transfer learning
- Better training techniques
- Higher-quality hardware
- Improved loss functions
- Better object-tracking methods
- Model optimization
- Modern deep-learning techniques

The experiment asks students to discuss recent advancements and improvements in deep learning and computer vision.

Result

A deep learning-based object detection system was successfully implemented using a pre-trained YOLO model. The system detected and localized multiple objects in an input image using bounding boxes, class labels, and confidence scores.